Plataforma académica multi-universidad

Documento de arquitectura

Alberto Alejandro Cortez

Abril 2026

Índice

1 Plataforma académica multi-universidad **2**
1.1 Documento de arquitectura . . . 2
1.2 Resumen ejecutivo . . . 2
1.3 Contexto y alcance . . . 3
1.3.1 Problema a resolver . . . 3
1.3.2 Alcance de la plataforma . . . 3
1.3.3 Fuera de alcance . . . 3
1.3.4 Usuarios objetivo . . . 3
1.4 Visión arquitectónica . . . 4
1.4.1 Plano académico-operacional . . . 4
1.4.2 Plano pedagógico-evaluativo . . . 5
1.4.3 Comunicación entre planos . . . 5
1.5 Requisitos . . . 5
1.5.1 Requisitos funcionales por rol . . . 5
1.5.2 Requisitos no funcionales . . . 6
1.5.3 Supuestos y restricciones . . . 7
1.6 Decisiones arquitectónicas . . . 7
1.6.1 ADR 1 — Multi-tenancy por Row-Level Security . . . 7
1.6.2 ADR 2 — Keycloak como IAM central con federación . . . 7
1.6.3 ADR 3 — Separación de bases lógicas . . . 7
1.6.4 ADR 4 — AI Gateway propio . . . 9
1.6.5 ADR 5 — Redis Streams como bus de eventos . . . 9
1.7 Modelo de dominio . . . 9
1.7.1 Jerarquía institucional . . . 9
1.7.2 Jerarquía operativa . . . 11
1.7.3 Jerarquía pedagógica . . . 11
1.7.4 Integridad referencial y scope . . . 11
1.8 Seguridad y autorización . . . 11
1.8.1 Autenticación . . . 12
1.8.2 Autorización . . . 12
1.8.3 Pseudonimización y protección de datos . . . 13
1.8.4 Auditoría de cambios de configuración . . . 13
1.9 Backend: servicios y contratos . . . 13
1.9.1 Servicios del plano académico . . . 13
1.9.2 Servicios del plano pedagógico . . . 15
1.9.3 Servicios transversales . . . 15
1.9.4 Contratos de eventos . . . 15
1.9.5 API REST: patrones y convenciones . . . 16
1.10 Frontend: aplicaciones y flujos . . . 16
1.10.1 Tres aplicaciones, una librería compartida . . . 16
1.10.2 Flujo central del estudiante . . . 18
1.10.3 Dashboard docente . . . 19
1.10.4 Stack y decisiones de frontend . . . 19
1.11 Integración de IA: tutor, RAG y generador . . . 19
1.11.1 Tutor socrático . . . 20

1.11.2 RAG por comisión . . . 20
1.11.3 Generador de ejercicios . . . 20
1.11.4 AI Gateway . . . 20
1.12 Datos: persistencia y estrategia . . . 21
1.12.1 Bases de datos . . . 21
1.12.2 Schemas clave . . . 21
1.12.3 Estrategia de particionado y archivado . . . 22
1.12.4 Backups y recuperación . . . 22
1.13 Observabilidad y operación . . . 23
1.13.1 Stack de observabilidad . . . 23
1.13.2 SLO clave . . . 23
1.13.3 Dashboards operativos . . . 23
1.13.4 Runbook de incidentes . . . 24
1.13.5 Deployment y ambientes . . . 24
1.14 Plan de desarrollo por fases . . . 24
1.14.1 F0 — Fundaciones (meses 1-2) . . . 24
1.14.2 F1 — Dominio académico (meses 3-4) . . . 26
1.14.3 F2 — Contenido y RAG (meses 5-6) . . . 26
1.14.4 F3 — Motor pedagógico (meses 7-10) — **núcleo de la tesis** . . . 26
1.14.5 F4 — Evaluación y analítica (meses 11-12) . . . 27
1.14.6 F5 — Integración multi-tenant real (meses 13-14) . . . 27
1.14.7 F6 — Hardening y pilotaje institucional (meses 15-16) . . . 27
1.14.8 Continuidad post-F6 . . . 28
1.15 Riesgos y mitigaciones . . . 28
1.15.1 Explosión de complejidad del clasificador . . . 28
1.15.2 Drift del prompt del tutor entre universidades . . . 28
1.15.3 Costo descontrolado de IA . . . 29
1.15.4 Otros riesgos a monitorear . . . 29
1.16 Glosario . . . 29
1.17 Apéndice: ADRs completos . . . 30
1.18 Documentos relacionados . . . 30

1 Plataforma académica multi-universidad

1.1 Documento de arquitectura

Versión 1.0.0 — Abril 2026 **Autor** Alberto Alejandro Cortez **Contexto** Operacionalización institucional del sistema AI-Native N4 **Relación con la tesis** Este documento describe la plataforma que aloja y escala el sistema diseñado en la tesis doctoral *"Modelo AI-Native con Trazabilidad Cognitiva N4 para la Formación en Programación Universitaria"*

1.2 Resumen ejecutivo

Este documento describe la arquitectura de una plataforma académica centralizada, multi-universidad y multi-tenant, diseñada para alojar el sistema AI-Native N4 de la tesis doctoral a escala institucional. La plataforma integra gestión académica tradicional (universidades, carreras, materias, comisiones, inscripciones, rúbricas, dashboards) con un núcleo pedagógico-evaluativo basado en trazabilidad cognitiva criptográficamente auditable, tutor socrático con prompt versionado y clasificador N4 validado estadísticamente.

El principio arquitectónico rector es la **separación en dos planos** mediante un bus de eventos: un plano académico-operacional, con tecnologías maduras y patrones consolidados, y un plano pedagógico-evaluativo, que es donde viven las propiedades auditables exigidas por la tesis. Esta separación permite que el plano académico evolucione con prácticas estándar de ingeniería sin contaminar las propiedades criptográficas y metodológicas del núcleo de investigación.

El plan de desarrollo abarca siete fases a lo largo de dieciséis meses, con hitos verificables al cierre de cada fase y un rollout piloto en dos universidades al finalizar. El esfuerzo está dimensionado para ser ejecutable por un equipo reducido dirigido por el doctorando, sin requerir inversión inicial en infraestructura de escala empresarial.

1.3 Contexto y alcance

1.3.1 Problema a resolver

Las instituciones universitarias carecen hoy de infraestructura que les permita incorporar asistentes de inteligencia artificial generativa en la formación programática preservando las propiedades que la evaluación educativa exige: trazabilidad del proceso, auditabilidad de las clasificaciones, validez estadística y aislamiento institucional. Los productos comerciales existentes (LMS, tutores IA genéricos, plataformas de ejercicios) resuelven aspectos parciales pero no la integración: o no tienen IA, o tienen IA sin trazabilidad, o tienen trazabilidad sin validación metodológica.

La tesis doctoral que fundamenta este desarrollo aporta el modelo pedagógico (N4), la arquitectura del núcleo evaluativo (CTR, tutor socrático versionado, clasificador con tres dimensiones de coherencia) y el protocolo de validación (Kappa de Cohen sobre muestras estratificadas). Lo que falta es el **envoltorio institucional** que permita desplegar ese núcleo en múltiples universidades simultáneamente, con roles diferenciados, gestión académica operativa, integración con sistemas institucionales existentes y auditabilidad por tenant.

1.3.2 Alcance de la plataforma

La plataforma provee:

Gestión académica multi-institucional. Creación y administración de universidades, facultades, carreras, planes de estudio, materias, comisiones, períodos lectivos e inscripciones. Importación masiva desde sistemas universitarios de información (SIS). Federación de identidad con IdPs institucionales (SAML, OIDC).

Experiencia del estudiante. Entorno integrado con editor de código, ejecución de tests, panel de tutor socrático con streaming y visualización de su propio proceso de aprendizaje. Acceso restringido a su comisión y sus datos personales.

Herramientas del docente. Carga de material de cátedra, gestión de base de conocimiento con RAG, generación asistida de ejercicios mediante IA, creación y edición de rúbricas, corrección asistida por el sistema, dashboards de seguimiento por comisión y por estudiante.

Panel de administración docente. Gestión completa de la estructura institucional de una universidad (carreras, materias, comisiones), configuración de presupuestos de IA, gestión global de permisos y auditoría institucional.

Núcleo pedagógico-evaluativo. Tutor socrático con prompt versionado, CTR (Cognitive Trace Record) con cadena SHA-256, clasificador N4 con tres dimensiones de coherencia y validación Kappa. Este núcleo es exactamente el descrito en la tesis doctoral, escalado para operar en contexto multi-tenant.

1.3.3 Fuera de alcance

Esta versión no incluye: aplicación móvil nativa (se provee web responsive con PWA), integración con plataformas de videoconferencia, generación de certificados digitales, pagos o facturación, ni gestión de infraestructura de exámenes presenciales con proctoring. Todos estos son candidatos naturales para versiones posteriores.

1.3.4 Usuarios objetivo

Tres roles primarios definen la mayor parte de la funcionalidad:

- **Docente administrador** (un responsable por universidad): configura la estructura académica completa, gestiona permisos, controla presupuestos de IA, supervisa auditoría institucional.
- **Docente**: opera dentro del alcance de sus comisiones; genera material, configura ejercicios y rúbricas, corrige y analiza.
- **Estudiante**: resuelve problemas con apoyo del tutor socrático y visibilidad sobre su propio proceso de aprendizaje.

Un cuarto rol transversal, **superadministrador**, opera a nivel global de plataforma y normalmente es ejercido por el equipo técnico.

1.4 Visión arquitectónica

La plataforma se organiza en dos planos funcionales desacoplados mediante un bus de eventos. El desacople es deliberado y es el principio arquitectónico central: permite que la funcionalidad académica tradicional (CRUDs, validaciones, reportes) evolucione con prácticas de ingeniería estándar sin contaminar las propiedades criptográficas y metodológicas del núcleo de investigación.

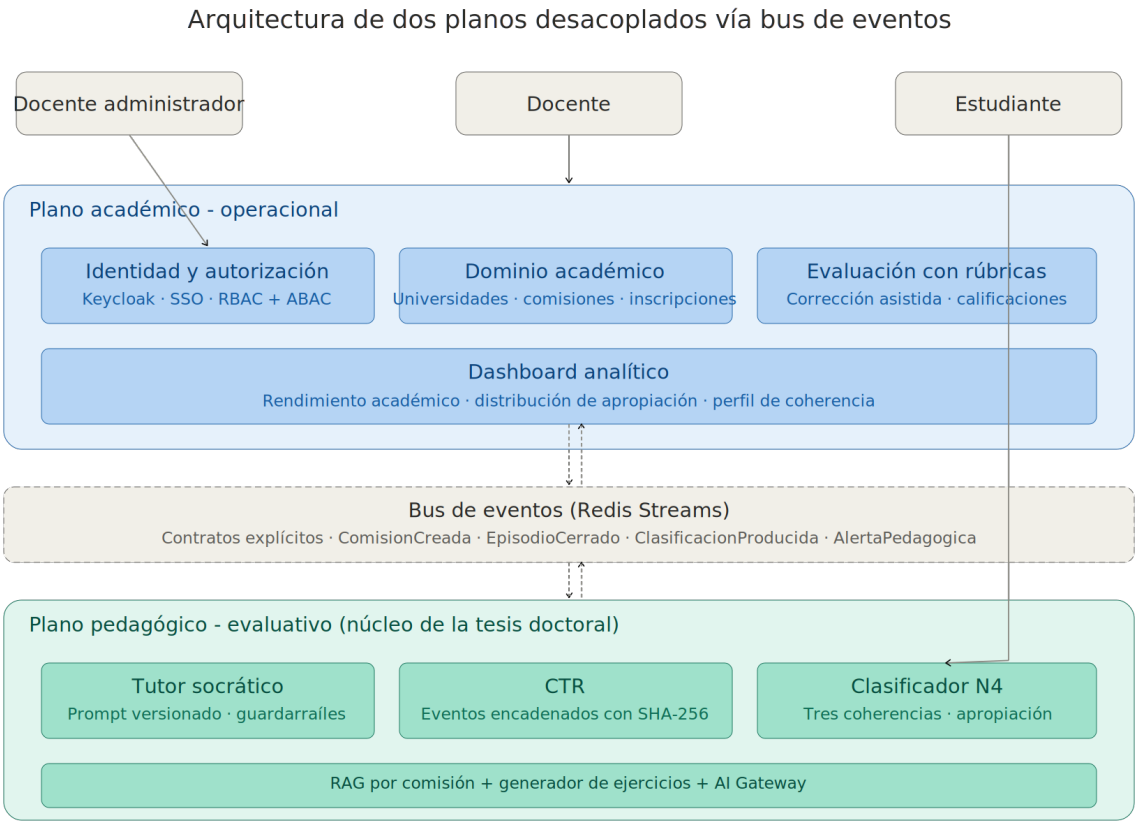

Figura 1: Arquitectura de dos planos desacoplados vía bus de eventos: el plano académico-operacional contiene identidad, dominio académico, evaluación y dashboards. El plano pedagógico-evaluativo contiene el tutor socrático, el CTR, el clasificador N4 y el RAG. Ambos se comunican exclusivamente por el bus de eventos con contratos explícitos, nunca por acceso directo a la base del otro.

1.4.1 Plano académico-operacional

Este plano aloja lo que un sistema universitario tradicional resuelve: identidad y autorización con Keycloak y federación institucional, modelado del dominio académico (universidades, carreras, materias, comisiones, inscripciones), evaluación con rúbricas y corrección asistida, y dashboards con métricas de rendimiento. Tecnologías maduras, patrones documentados,

riesgo de diseño bajo. La complejidad aquí es de **volumen** (muchas entidades, muchas relaciones, muchos permisos) pero no de **naturaleza** (no hay problemas no resueltos).

1.4.2 Plano pedagógico-evaluativo

Este plano aloja el núcleo de la tesis: tutor socrático con prompt versionado criptográficamente, CTR con cadena SHA-256 encadenada, clasificador N4 con tres dimensiones de coherencia (temporal, código-discurso, inter-iteración) y validación estadística Kappa contra anotadores humanos. La complejidad aquí es de **naturaleza**: hay decisiones de diseño sin precedente comercial claro, requiere validación metodológica explícita, y las propiedades auditables son condición de aceptabilidad académica.

El diseño detallado de este plano está en el documento técnico complementario *Sistema AI-Native N4 — Guía de implementación técnica*, que describe el CTR, el tutor, los tres analizadores de coherencia, el clasificador de apropiación, el protocolo Kappa y la calibración del `reference_profile`. Este documento de arquitectura se refiere a ese núcleo como una unidad y se concentra en cómo integrarlo en la plataforma multi-tenant.

1.4.3 Comunicación entre planos

Los dos planos se comunican exclusivamente a través del bus de eventos, con contratos explícitos versionados. El plano académico emite eventos como `ComisionCreada`, `EstudianteInscripto`, `MaterialCargado`; el plano pedagógico los consume para scopear el tutor, el RAG y el CTR. A la inversa, el plano pedagógico emite `EpisodioCerrado`, `ClasificacionProducida`, `AlertaPedagogica`; el plano académico los consume para alimentar dashboards y notificaciones. Ningún servicio lee directamente la base de datos del otro plano.

Esta comunicación unidireccional por eventos tiene dos propiedades valiosas: preserva la integridad criptográfica del CTR (no hay queries analíticas compitiendo con la escritura de eventos por lock) y permite evolucionar cada plano independientemente siempre que el contrato de eventos se mantenga o evolucione hacia atrás compatible.

1.5 Requisitos

1.5.1 Requisitos funcionales por rol

Docente administrador

Configuración institucional: crear y editar universidades, facultades y carreras bajo su ámbito; dar de alta materias con sus objetivos pedagógicos; crear comisiones por período lectivo; asignar docentes a comisiones; importar inscripciones masivas desde CSV o integración directa con el SIS institucional.

Gestión global de permisos: definir qué docentes tienen acceso a qué comisiones; gestionar roles secundarios (asistente, revisor); configurar presupuestos de IA por carrera o comisión; acceder a auditoría institucional.

Supervisión: acceder a dashboards agregados de la universidad; revisar alertas pedagógicas escaladas; aprobar cambios mayores en el prompt del tutor (MAJOR versions); acceder al registro de cambios de configuración.

Docente

Autoría de contenido: cargar material de cátedra en múltiples formatos (PDF, Markdown, código ZIP, video); gestionar la base de conocimiento de sus comisiones; generar ejercicios mediante prompts de IA con preview estructurado y edición; crear actividades evaluativas con calendario y condiciones de entrega.

Diseño evaluativo: crear y editar rúbricas por comisión o reutilizarlas entre comisiones; definir criterios ponderados con niveles de logro; asociar rúbricas a actividades; configurar umbrales de alerta pedagógica específicos.

Seguimiento y corrección: acceder al dashboard con vista por comisión y por estudiante; revisar alertas (patrones sostenidos de delegación, intentos de jailbreak, progresiones

positivas); utilizar la corrección asistida que pre-llena puntajes a partir del CTR; publicar calificaciones finales.

Restricciones del docente: no puede crear universidades, carreras, materias ni comisiones; no puede modificar el prompt del sistema (solo proponer cambios); no puede ver datos de comisiones que no tiene asignadas.

Estudiante

Participación: ver sus actividades pendientes, iniciar episodios de resolución, conversar con el tutor socrático, escribir y ejecutar código, ejecutar casos de prueba, entregar solución.

Autorregulación: acceder a su propia vista "Mi proceso" con visualización pedagógica de su CTR y perfil de coherencia; ver retroalimentación formativa automática; recibir alertas cuando el sistema detecta patrones adversos.

Tutor socrático (sistema)

Guía interactiva durante la resolución siguiendo las reglas del prompt versionado (detección de jailbreak, clasificación epistemológica del prompt del estudiante, respuesta socrática streaming, autoevaluación posterior). Registro completo en el CTR de cada evento (prompts enviados, respuestas recibidas, ediciones de código, tests ejecutados, debugger usado) con encadenamiento SHA-256.

Evaluación docente (sistema)

Corrección asistida: el sistema sugiere puntajes por criterio de rúbrica basándose en el código entregado, los tests pasados y la evidencia del CTR; el docente confirma o ajusta cada puntaje; la calificación final se almacena con trazabilidad al CTR que la sustenta. Contexto académico: toda corrección se inserta en la jerarquía universidad → carrera → materia → comisión para permitir análisis agregados.

Analítica educativa (sistema)

Dashboard docente con tres vistas complementarias no colapsables: métricas de rendimiento tradicional (notas, tiempo invertido, porcentaje de completitud), distribución de apropiación (delegación pasiva, superficial, reflexiva), y perfil de coherencia vectorial (las tres dimensiones mostradas por separado). Análisis por comisión con agregados visuales y exportación para actas.

1.5.2 Requisitos no funcionales

Arquitectura escalable con enfoque multi-tenant. Cada universidad es un tenant aislado; el diseño soporta desde una única universidad en modo single-tenant hasta decenas de universidades en modo multi-tenant con aislamiento garantizado criptográficamente a nivel de base de datos.

Seguridad. Protección de datos personales con pseudonimización por defecto; control de acceso mediante combinación RBAC (roles) + ABAC (atributos como comisión y carrera); cifrado en tránsito (TLS 1.3) y en reposo para datos sensibles; gestión de secretos en vault dedicado; cumplimiento con marcos de protección de datos educativos.

Auditoría. Registro append-only de todos los eventos del CTR con integridad criptográfica verificable; registro de todas las clasificaciones con su `config_hash` para reconstrucción histórica; registro de cambios de configuración (prompt, reference_profile, políticas) en repositorio Git con commits firmados GPG; anclaje externo del hash final de cada episodio en log institucional append-only.

Rendimiento. Latencia del tutor socrático menor a 2 segundos para el primer token, menor a 5 segundos para respuesta completa del 95º percentil; escritura al CTR no bloqueante para el estudiante; dashboards con latencia menor a 3 segundos sobre agregados de hasta 10.000 episodios; SLO de disponibilidad del 99.5% en el primer año de operación.

Observabilidad. Métricas por tenant, por carrera y por feature; alertas automáticas ante degradación de SLO; trazas distribuidas de extremo a extremo; logs estructurados con correlation id.

1.5.3 Supuestos y restricciones

La plataforma asume acceso a al menos un proveedor de LLM comercial (Anthropic como primario, con OpenAI y providers locales como respaldo). Asume infraestructura mínima de una VM con 8 vCPU y 32 GB RAM para operar hasta 500 estudiantes concurrentes. Asume disponibilidad de un IdP institucional compatible con SAML u OIDC para cada universidad que se integra.

La plataforma no asume disponibilidad de Kubernetes en el pilotaje inicial (Docker Compose basta); sí asume disponibilidad de Kubernetes para despliegues productivos con más de una universidad activa.

1.6 Decisiones arquitectónicas

Las decisiones arquitectónicas clave se documentan como ADRs (Architecture Decision Records). El repositorio mantiene un directorio `docs/adr/` con un archivo por decisión en formato MADR. A continuación se presentan las cinco decisiones fundamentales que condicionan el resto del diseño.

1.6.1 ADR 1 — Multi-tenancy por Row-Level Security

Contexto. La plataforma debe aislar los datos de cada universidad de forma que un bug en código de aplicación no pueda filtrar información entre tenants. Tres opciones principales: schema-per-tenant, database-per-tenant, Row-Level Security (RLS) con columna `tenant_id`.

Decisión. Usar RLS de PostgreSQL con columna `tenant_id` en toda tabla relevante. Cada request de la aplicación inyecta el `tenant_id` extraído del JWT como `SET app.current_tenant` al inicio de la transacción. Las políticas RLS filtran automáticamente toda query para que solo devuelva filas del tenant activo. Se reserva la capacidad de "graduar" un tenant grande o con requisitos de compliance a database dedicada sin cambiar código de aplicación.

Consecuencias positivas. Aislamiento en la capa de base de datos, no de aplicación. Operación simplificada (una sola base a mantener). Migraciones atómicas para todos los tenants. Facilidad de queries analíticas cross-tenant para el `superadmin`.

Consecuencias negativas. Requiere rigor en las políticas RLS (un bug en una política afecta a todos). Performance de queries complejas puede degradarse si los índices no consideran `tenant_id`. Tests automatizados deben verificar que toda tabla tiene policy activa.

1.6.2 ADR 2 — Keycloak como IAM central con federación

Contexto. Cada universidad puede tener su propio IdP institucional (LDAP, Google Workspace, SAML). La plataforma necesita autenticación federada, gestión de roles, y autorización fine-grained a nivel de comisión específica.

Decisión. Keycloak como IAM central, con un realm por universidad. Cada realm federa con el IdP institucional mediante SAML u OIDC. Los roles base (`superadmin`, `docente_admin`, `docente`, `estudiante`) se modelan como realm-roles. La autorización fine-grained (acceso a una comisión específica) se implementa con **Casbin** en la capa de aplicación, porque RBAC puro no alcanza: un usuario es "docente" pero solo de ciertas comisiones.

Consecuencias positivas. Estándar maduro con soporte de SSO y federación. Cada universidad mantiene su IdP institucional sin cambios. Tokens JWT autocontenidos permiten validación sin round-trip a Keycloak.

Consecuencias negativas. Dependencia de Keycloak; su disponibilidad afecta a toda la plataforma. Políticas Casbin deben mantenerse sincronizadas con los estados de comisión activa.

1.6.3 ADR 3 — Separación de bases lógicas

Contexto. El plano académico y el plano pedagógico tienen requisitos distintos: el académico es transaccional clásico con muchas escrituras puntuales y lecturas complejas; el pedagógico tiene escrituras masivas append-only (eventos CTR) con queries analíticas batch.

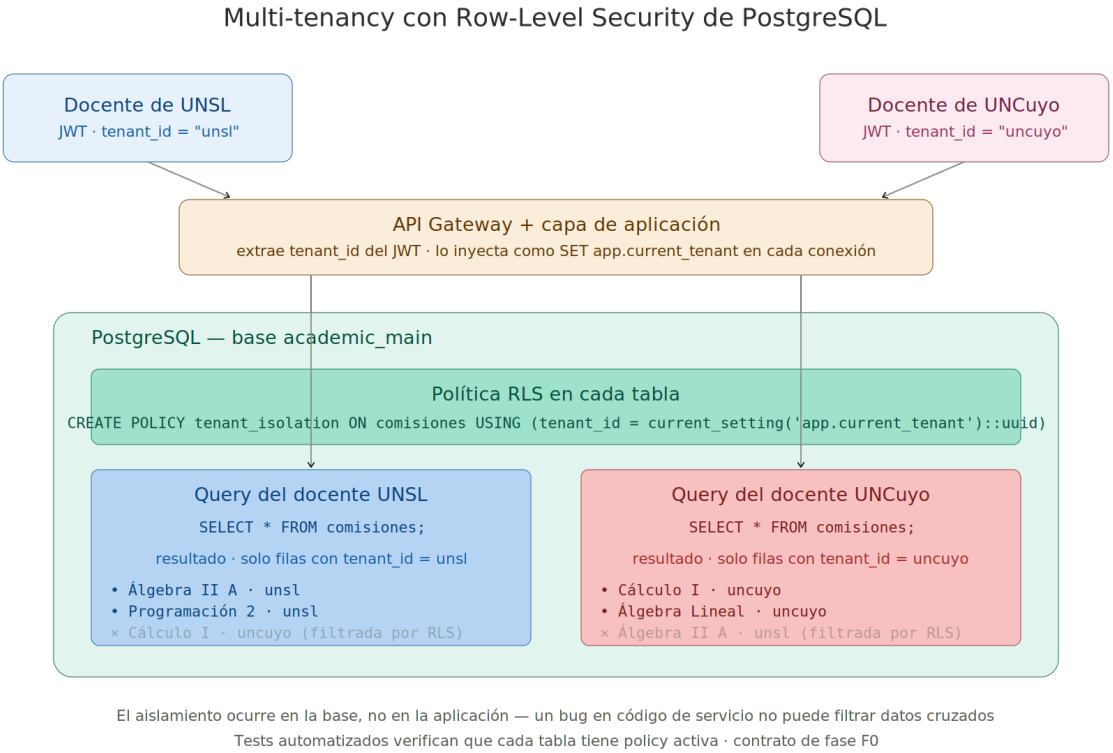

Figura 2: Diagrama de multi-tenancy con Row-Level Security: un docente de UNSL y otro de UNCuyo hacen la misma consulta SELECT sobre la tabla de comisiones; la política RLS de PostgreSQL filtra automáticamente los resultados según el tenant_id extraído del JWT y seteado con SET app.current_tenant, de modo que cada docente solo ve las filas de su universidad.

Decisión. Dos bases lógicas separadas en el mismo cluster PostgreSQL: `academic_main` para el plano académico y `ctr_store` para el CTR y clasificaciones. Roles de PostgreSQL distintos con permisos distintos: el backend del plano académico no puede escribir en `ctr_store`, y el servicio CTR no puede leer fuera de su base. Una tercera base `identity_store` aloja el mapping pseudónimo → identidad real con acceso aún más restringido.

Consecuencias positivas. Un compromiso de credenciales en un plano no expone el otro. Queries analíticas largas en `ctr_store` no bloquean operaciones del plano académico. VACUUM y operaciones de mantenimiento se pueden programar independientemente.

Consecuencias negativas. Tres conjuntos de conexiones, backups y réplicas a administrar. Joins cross-base requieren llevar datos a la aplicación.

1.6.4 ADR 4 — AI Gateway propio

Contexto. La plataforma invoca LLMs desde cuatro servicios distintos (tutor, clasificador, content, evaluation) para propósitos diferentes. Cada universidad tiene presupuesto mensual y necesita observabilidad de costos. Las credenciales de API no pueden estar distribuidas en todos los servicios.

Decisión. Un servicio dedicado `ai-gateway` que centraliza todas las invocaciones a LLMs y embeddings. Ningún otro servicio habla con Anthropic, OpenAI o Voyage directamente. El gateway maneja autenticación del request, chequeo de presupuesto por tenant, rate limiting, routing entre modelos, circuit breakers, retries, logging de tokens consumidos imputados al tenant, y caché para queries idempotentes.

Consecuencias positivas. Observabilidad centralizada por tenant, carrera y feature. Posibilidad de fallback entre proveedores. Credenciales de API en un solo lugar. Presupuestos y kill-switches efectivos. Caché unificado.

Consecuencias negativas. Punto central que debe ser altamente disponible. Latencia adicional (típicamente 10-30 ms) por el hop intermedio. Otra pieza de software a mantener.

1.6.5 ADR 5 — Redis Streams como bus de eventos

Contexto. La comunicación entre planos y entre servicios requiere un bus de eventos con garantías at-least-once, orden estricto por clave de partición (episode_id), y observabilidad de backpressure.

Decisión. Redis Streams como bus para el MVP y las primeras fases productivas. Sharding por `hash(episode_id) mod 8` para garantizar orden por episodio con paralelismo entre episodios distintos. Consumer groups con acknowledgment manual después del commit transaccional. DLQ (`ctr.dead`) para eventos que fallan tras tres reintentos. Migración planificada a Apache Kafka solo cuando el volumen justifique el costo operativo adicional (típicamente, más de tres universidades con alto volumen concurrente).

Consecuencias positivas. Simplicidad operativa (Redis ya está en el stack para caché). Garantías suficientes para el pilotaje y producción temprana. Curva de aprendizaje baja.

Consecuencias negativas. Redis Streams no está diseñado para retención larga; eventos antiguos se purgan. Límite práctico alrededor de 10.000 eventos/segundo por stream. Migración futura a Kafka requerirá rediseño de consumidores.

1.7 Modelo de dominio

El dominio se organiza en tres jerarquías que deben modelarse con precisión porque los permisos, el scope de los recursos y los filtros de dashboards dependen de cómo se expresen las relaciones entre entidades.

1.7.1 Jerarquía institucional

Modela la estructura académica formal:

```
Universidad → Facultad (opcional) → Carrera → Plan de estudios → Materia
```

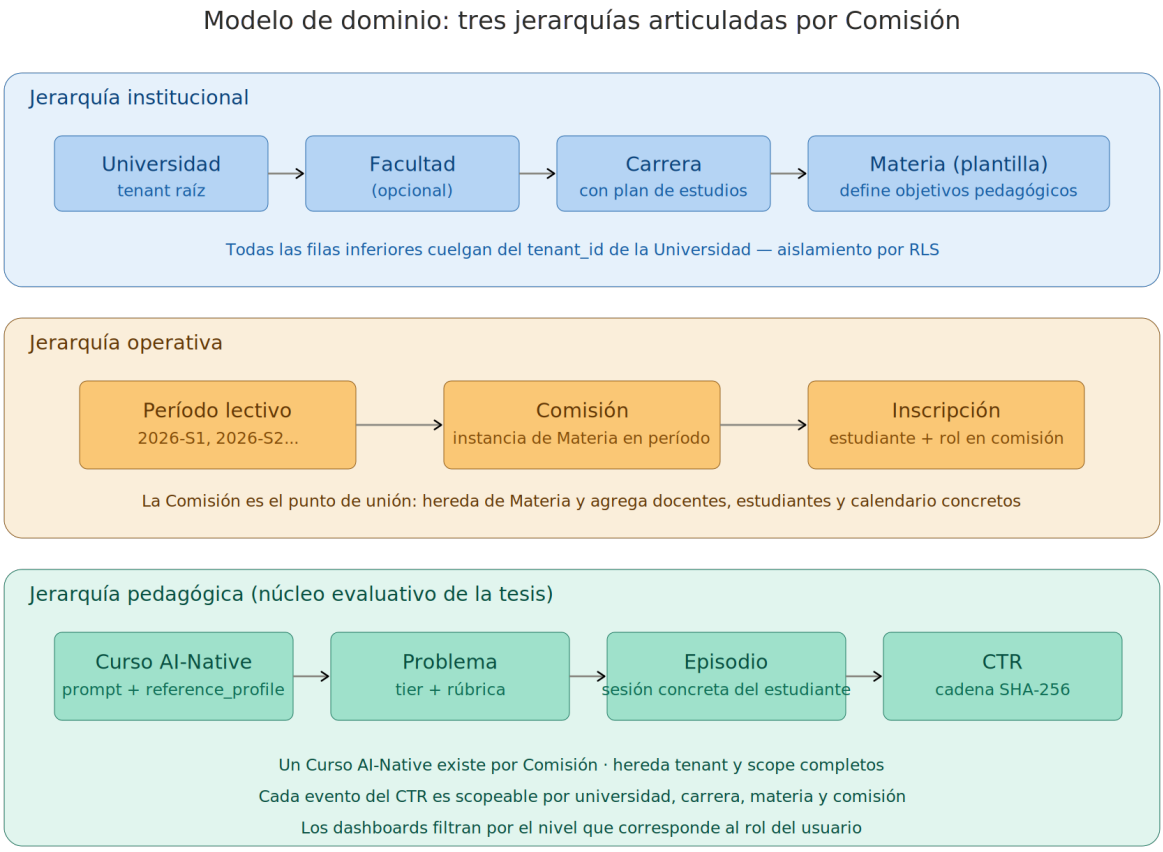

Figura 3: Modelo de dominio con tres jerarquías articuladas: la jerarquía institucional va de universidad a materia pasando por facultad y carrera; la jerarquía operativa combina período lectivo y comisión para producir instancias concretas; la jerarquía pedagógica del núcleo de la tesis va de curso AI-Native a CTR pasando por problema y episodio. La entidad Comisión es el punto de unión donde lo académico se vuelve pedagógico.

La **Universidad** es el tenant raíz: toda tabla descendiente lleva su `tenant_id` y está protegida por política RLS. La **Facultad** es opcional porque algunas universidades pequeñas no tienen esa subdivisión. La **Carrera** contiene múltiples **Planes de estudio** versionados (los estudiantes ingresantes en años distintos pueden cursar planes diferentes). La **Materia** es una plantilla con objetivos pedagógicos, prerrequisitos y correlatividades.

1.7.2 Jerarquía operativa

Modela las instancias concretas que un docente y un estudiante experimentan:

`Universidad → Período lectivo (2026-S1) → Comisión → Inscripción`

La distinción entre **Materia** (plantilla atemporal) y **Comisión** (instancia en un período) es crucial. *Álgebra II* es una materia que existe durante años; la *Comisión 2026-S1-Álgebra-II-A* es su instancia específica con sus docentes, estudiantes matriculados, calendario, material de cátedra y configuración pedagógica propia. El tutor socrático, el CTR y el RAG viven al nivel de comisión, no de materia — porque el prompt y el `reference_profile` pueden evolucionar cuatrimestre a cuatrimestre sin perder la auditabilidad histórica.

La **Inscripción** es la relación muchos-a-muchos entre estudiante y comisión, con rol (estudiante regular, oyente, reinscripción) y estado (activo, cursando, aprobado, desaprobado, abandonado).

1.7.3 Jerarquía pedagógica

Es donde vive el núcleo de la tesis:

`Comisión → Curso AI-Native → Problema → Episodio → CTR`

El **Curso AI-Native** es el despliegue del sistema para una comisión específica, con su prompt del sistema (heredado del prompt global por MAJOR, con ajustes MINOR por comisión si corresponde), su `reference_profile` por tier de problema, y su configuración del clasificador. Cada Curso tiene un `config_hash` que forma parte del hash de cada evento del CTR, garantizando reproducibilidad.

El **Problema** es una unidad de trabajo: enunciado, casos de prueba, rúbrica asociada, tier (cs1_easy, cs1_medium, cs2), metadata de técnica algorítmica. Los problemas pueden generarse con IA (el docente usa el generador de ejercicios del sistema) o cargarse manualmente.

El **Episodio** es una sesión concreta de un estudiante resolviendo un problema. Tiene inicio (`EpisodioAbierto`) y fin (`EpisodioCerrado`), y entre esos dos eventos se encadenan todos los demás eventos del CTR.

El **CTR** (Cognitive Trace Record) es la cadena criptográfica completa de eventos del episodio. Su diseño está detallado en el documento técnico complementario.

1.7.4 Integridad referencial y scope

Toda entidad del dominio lleva explícitamente el `tenant_id` de la Universidad, aun cuando podría inferirse por transitividad. Esto es redundancia intencional: hace que las políticas RLS sean triviales de escribir y que los índices compuestos `(tenant_id, ...)` sean óptimos para las queries más frecuentes.

Para los dashboards que requieren filtros por universidad, carrera, materia o comisión, la query analítica usa los identificadores de cada nivel sin necesidad de joins: la vista materializada `episode_summary` incluye columnas `tenant_id`, `carrera_id`, `materia_id`, `comision_id` como denormalización explícita. La estrategia de actualización de esa vista se describe en la sección 11.

1.8 Seguridad y autorización

La seguridad de la plataforma se construye en tres capas que se refuerzan mutuamente: autenticación federada con Keycloak, autorización por políticas (RBAC + ABAC) con Casbin, y aislamiento de datos en la base (RLS). Ningún bug en una capa compromete a las otras.

1.8.1 Autenticación

Cada universidad opera como un **realm** independiente en Keycloak. El realm puede autenticar localmente (usuarios creados en la plataforma) o federarse con un IdP institucional mediante SAML, OIDC o LDAP. La federación es transparente para el usuario: escribe su correo institucional, es redirigido al IdP, vuelve con un token Keycloak.

El token JWT emitido por Keycloak contiene claims estándar (`sub`, `email`, `exp`) y claims custom de la plataforma (`tenant_id`, `realm`, `roles`, `comisiones_activas`). Los servicios backend validan el JWT en cada request usando las claves públicas de Keycloak (rotadas automáticamente). El `tenant_id` se extrae del JWT y se inyecta como configuración de sesión PostgreSQL antes de toda query.

1.8.2 Autorización

La matriz de permisos del sistema se materializa en políticas Casbin. El modelo Casbin utiliza cuatro dimensiones: sujeto (usuario), dominio (tenant), objeto (recurso) y acción. La sintaxis permite `keyMatch` para expresar jerarquías de recursos (`comision:*` contiene `comision:123`).

Matriz consolidada de permisos por rol:

Recurso	Acción	superadmin	docente_admin	docente	estudiante
Universidad	crear / editar	✓ global	✓ propia	—	—
Facultad / Carrera	crear / editar	✓ global	✓ propia univ.	—	—
Materia (plantilla)	crear / editar	✓ global	✓ propia univ.	—	—
Comisión	crear / asignar docentes	✓ global	✓ propia univ.	—	—
Inscripciones	importar / listar	✓ global	✓ propia univ.	✓ sus comisiones	✓ solo la propia
Prompt del sistema (versiones)	publicar MAJOR	✓ global	✓ propia univ.	—	—
Prompt del sistema (versiones)	publicar MINOR	✓ global	✓ propia univ.	✓ propuesta	—
Material de cátedra	cargar / editar	✓ global	✓ propia univ.	✓ sus comisiones	—
Base de conocimiento (RAG)	gestionar	✓ global	✓ propia univ.	✓ sus comisiones	—
Ejercicios (generador IA)	generar	✓ global	✓ propia univ.	✓ sus comisiones	—
Actividad evaluativa	crear / publicar	✓ global	✓ propia univ.	✓ sus comisiones	—
Rúbricas	crear / asignar	✓ global	✓ propia univ.	✓ sus comisiones	—
Tutor socrático	conversar	✓ auditoría	✓ auditoría	✓ sus comisiones	✓ sus comisiones
CTR	leer	✓ global	✓ propia univ.	✓ sus alumnos	✓ el propio
Dashboard analítico	ver agregado	✓ global	✓ propia univ.	✓ sus comisiones	✓ vista propia
Corrección (rúbricas)	evaluar	—	—	✓ sus alumnos	—
Calificaciones finales	publicar	—	✓ propia univ.	✓ sus alumnos	—

Recurso	Acción	superadmin	docente_admin	docente	estudiante
Presupuesto IA	configurar	✓ global	✓ propia univ.	—	—

1.8.3 Pseudonimización y protección de datos

La plataforma opera internamente con `student_pseudonym` (UUID) como identificador persistente. El mapeo entre `student_pseudonym` y la identidad real (DNI, nombre, correo) vive en una base dedicada `identity_store` con acceso restringido al equipo de coordinación pedagógica. El backend del plano académico y del plano pedagógico nunca consulta esta base durante el flujo normal; solo la usan el `identity-service` durante el login y procesos administrativos específicos.

Todos los eventos del CTR, clasificaciones, clasificaciones de anotadores humanos y exportaciones analíticas usan solo `student_pseudonym`. Esto es compatible con el principio de la tesis de que el CTR pseudonimizado no es un dato personal en sentido legal.

1.8.4 Auditoría de cambios de configuración

El `governance-service` custodia todas las configuraciones que afectan la interpretación del CTR en un repositorio Git separado (`ai-native-prompts`). Cada cambio al prompt, al `reference_profile` o a los umbrales del clasificador es un commit firmado con GPG. La plataforma expone el hash activo de cada configuración en su endpoint `/health` y los servicios verifican al arrancar que el hash registrado coincide con el computado (fail-loud).

1.9 Backend: servicios y contratos

La partición en servicios privilegia **responsabilidad funcional** sobre tamaño de módulo. Diez servicios en total, agrupados por los dos planos y los transversales.

1.9.1 Servicios del plano académico

`identity-service`. Wrapper fino sobre la Admin API de Keycloak. Expone endpoints para gestión de usuarios (crear, activar, desactivar), asignación de roles, gestión de grupos por comisión, y consulta de federación con IdPs externos. No almacena credenciales propias; toda autenticación delega en Keycloak.

`academic-service`. CRUDs del dominio académico. Stack: FastAPI + SQLAlchemy 2.0 + Pydantic v2 + Alembic para migraciones. Expone REST con OpenAPI auto-generado. Emite eventos al bus cuando se crean/modifican comisiones, se asignan docentes, se crean problemas. Contiene la lógica de validación de correlatividades y prerrequisitos.

`enrollment-service`. Aislado del `academic-service` porque tiene integraciones externas complejas con sistemas institucionales (SIS). Maneja importación masiva de inscripciones por CSV, sincronización periódica con el SIS de cada universidad, y reconciliación de diferencias. Emite `EstudianteInscripto`, `EstudianteDesinscripto`, `CambioDeComision`.

`evaluation-service`. Rúbricas estructuradas en JSON Schema versionado; evaluación asistida que pre-llena puntajes a partir del CTR y las respuestas del estudiante; cálculo de calificaciones finales con propagación a acta. Consume eventos `ClasificacionProducida` y `EpisodioCerrado` del plano pedagógico para tener contexto al momento de corregir.

`analytics-service`. Consume eventos de ambos planos y mantiene tablas agregadas optimizadas para dashboards. Postgres con vistas materializadas refrescadas incrementalmente (`pg_cron`) para el MVP; migración a ClickHouse planificada si el volumen analítico justifica el costo operativo. Expone endpoints GraphQL para los dashboards (los filtros combinatorios son más naturales en GraphQL que en REST).

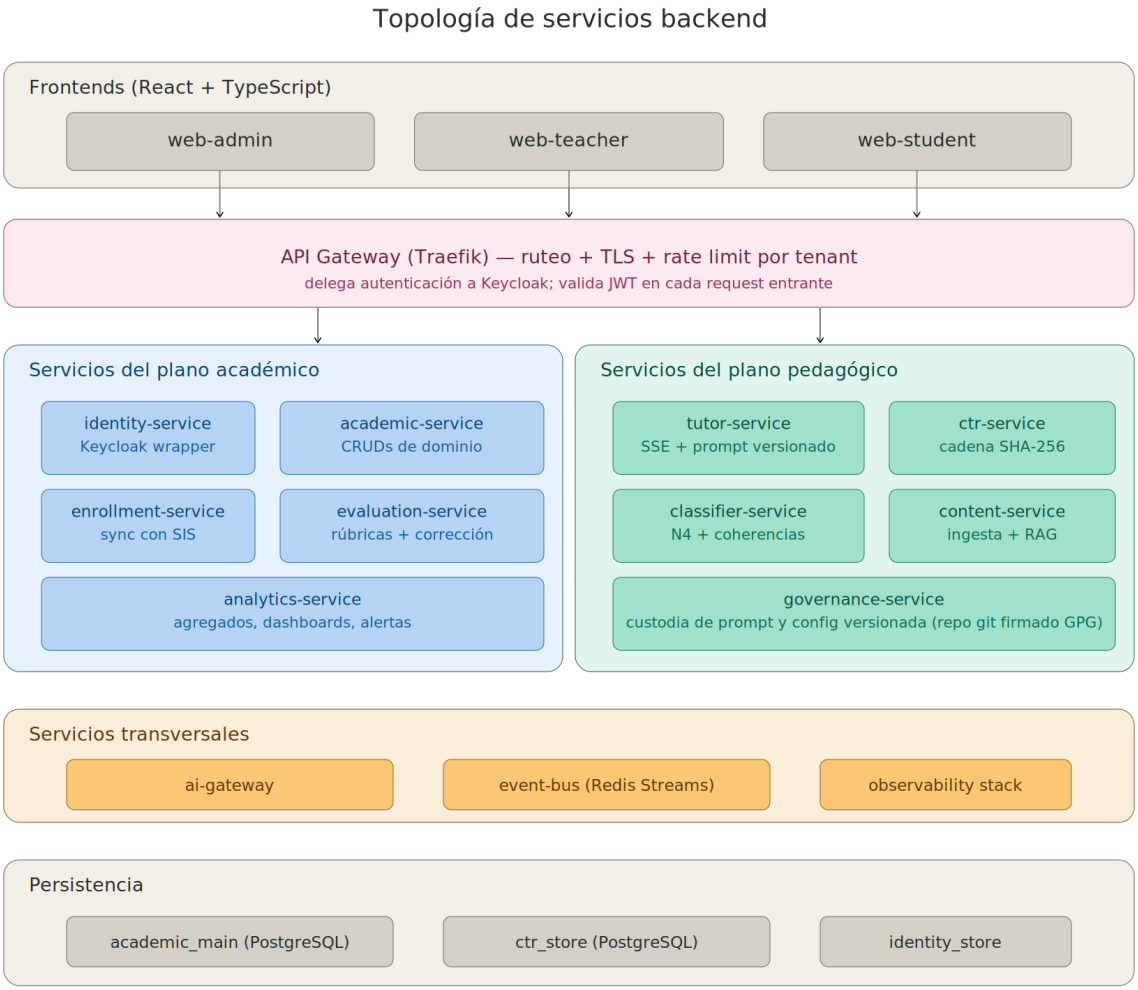

Figura 4: Topología de servicios backend: frontends React (web-admin, web-teacher, web-student) acceden a través de un API Gateway que delega autenticación a Keycloak. El gateway rutea a los servicios del plano académico (identity, academic, enrollment, evaluation, analytics) y a los servicios del plano pedagógico (tutor, CTR, classifier, content, governance). Un AI Gateway y un bus Redis Streams actúan como servicios transversales. Tres bases PostgreSQL separadas almacenan datos académicos, CTR e identidad.

1.9.2 Servicios del plano pedagógico

Los cuatro servicios pedagógicos están descritos en detalle en el documento *Sistema AI-Native N4 — Guía de implementación técnica*. Acá los menciono brevemente por completitud.

`tutor-service`. Servicio del tutor socrático con prompt versionado, streaming SSE, guardarraíles pre-check y post-check, clasificación de `prompt_kind` con LLM auxiliar. Write-only al CTR: publica eventos al bus pero nunca lee del CTR para decidir su respuesta.

`ctr-service`. Worker consumidor del bus de eventos. Calcula el `chain_hash` SHA-256 de cada evento, persiste en `ctr_store`, gestiona idempotencia por `event_uuid` único, administra DLQ para eventos que fallan persistentemente. Ocho réplicas del worker, una por partición del stream (`ctr.p0` a `ctr.p7`).

`classifier-service`. Consumidor de episodios cerrados. Ejecuta el etiquetador N1-N4 y los tres analizadores de coherencia (temporal, código-discurso, inter-iteración). Produce la clasificación de apropiación con árbol de decisión explicable. Las clasificaciones son append-only en tabla `classifications` con `classifier_config_hash`.

`content-service`. Ingesta de material de cátedra (PDF, Markdown, código, video con transcripción Whisper). Chunking semántico con tree-sitter (código) y unstructured.io (documentos). Embeddings con `multilingual-e5-large` o Voyage AI. Retrieval con filtro por `comision_id` y re-ranking con `bge-reranker-base`. Expone endpoint interno al `tutor-service` para inyección de contexto en prompts.

`governance-service`. Custodia del prompt versionado y del `classifier_config`. Implementado sobre un repositorio Git separado con commits firmados GPG. Expone el hash activo de cada configuración en `/health`. Todos los demás servicios llaman a este endpoint al arrancar y se niegan a operar si el hash registrado no coincide con el computado.

1.9.3 Servicios transversales

`ai-gateway`. Centraliza todas las invocaciones a LLMs y embeddings externos.

El gateway maneja cuatro proveedores (Anthropic como primario, OpenAI y Voyage como secundarios, Ollama/local como fallback), configura circuit breakers por proveedor, implementa caché para queries idempotentes (clasificación con temperatura 0) y expone métricas detalladas por tenant para observabilidad de costos.

`event-bus`. No es un servicio propio sino la infraestructura de Redis Streams con sus configuraciones de consumer groups, maxlen aproximado por stream, políticas de retención y monitoreo de lag.

`observability-stack`. OpenTelemetry Collector recibiendo traces, métricas y logs de todos los servicios. Jaeger para traces, Prometheus para métricas, Loki para logs, Grafana como frontend unificado, Sentry para errores de runtime. Dashboards predefinidos para los SLO clave.

1.9.4 Contratos de eventos

Los eventos que cruzan planos o servicios tienen contrato estable. La definición se mantiene en un repositorio compartido (`platform-contracts`) como schemas JSON Schema o Pydantic. Cambios de schema siguen versionado semántico:

Evento	Emite	Consume	Trigger	Payload principal
`UniversidadCreada`	academic	content, analytics	provisioning de tenant	tenant_id, nombre, config
`CarreraCreada`	academic	analytics	setup institucional	carrera_id, universidad_id
`ComisionCreada`	academic	content, tutor, ctr, classifier	apertura de cursada	comision_id, curso_config
`EstudianteInscripto`	enrollment	academic, identity	matrícula	pseudonym, comision_id

Evento	Emite	Consume	Trigger	Payload principal
`MaterialIngerido`	content	tutor	post-chunking	material_id, comision_id, chunks
`EpisodioAbierto`	tutor	ctr	inicio de sesión	episode_id, student, problem
`PromptEnviado`	tutor	ctr	pregunta al tutor	episode_id, content, prompt_kind
`RespuestaRecibida`	tutor	ctr	respuesta del tutor	episode_id, content, compliance
`EdicionCodigo`	web-student	ctr	edición del código	episode_id, snapshot, diff
`TestsEjecutados`	web-student	ctr	ejecución de tests	episode_id, results, passed
`EpisodioCerrado`	tutor / web-student	ctr, classifier	fin de sesión	episode_id, final_hash
`ClasificacionProducida`	classifier	evaluation, analytics	análisis diferido	episode_id, apropriation, coherencias
`AlertaPedagogica`	classifier	analytics, identity	umbrales	student, type, severity
`CalificacionPublicada`	evaluation	analytics, enrollment	corrección completa	activity_id, student, score

1.9.5 API REST: patrones y convenciones

Todos los servicios exponen API REST con OpenAPI. Las convenciones son:

- **Versionado en URL**: `/api/v1/comisiones/{id}`. Versionado semántico de la API misma.
- **Paginación por cursor**, no por offset, para estabilidad ante escrituras concurrentes.
- **Filtrado por query string** con operadores explícitos: `?estado=activa&carrera_id=uuid:eq&fecha:gte=2`
  `01-01`.
- **Campos seleccionables** con `?fields=id,nombre,docente` para minimizar payload.
- **Idempotencia por header `Idempotency-Key`** en operaciones de creación.
- **Respuestas estandarizadas** con `data`, `meta` (paginación), `errors` (formato problem+json).
- **HATEOAS moderado**: links a recursos relacionados cuando son descubribles, sin obsesión por HAL.

1.10 Frontend: aplicaciones y flujos

El frontend se separa en tres aplicaciones distintas, cada una optimizada para un conjunto de roles. No comparten sesión de navegador (cada una autentica contra Keycloak con su clientId), pero comparten una librería de componentes `@platform/ui` con los bloques de diseño comunes.

1.10.1 Tres aplicaciones, una librería compartida

`web-admin` — Para `superadmin` y `docente_admin`. Dominio: gestión institucional. Estética densa, tabular, funcional. Uso de `@tanstack/react-table` para tablas grandes con filtros y acciones bulk, `@tanstack/react-form` para formularios con validación. Las vistas principales son CRUDs sofisticados sobre universidades, carreras, materias, comisiones, usuarios, permisos, presupuestos. No optimizada para móvil (es una herramienta de trabajo que se usa en desktop).

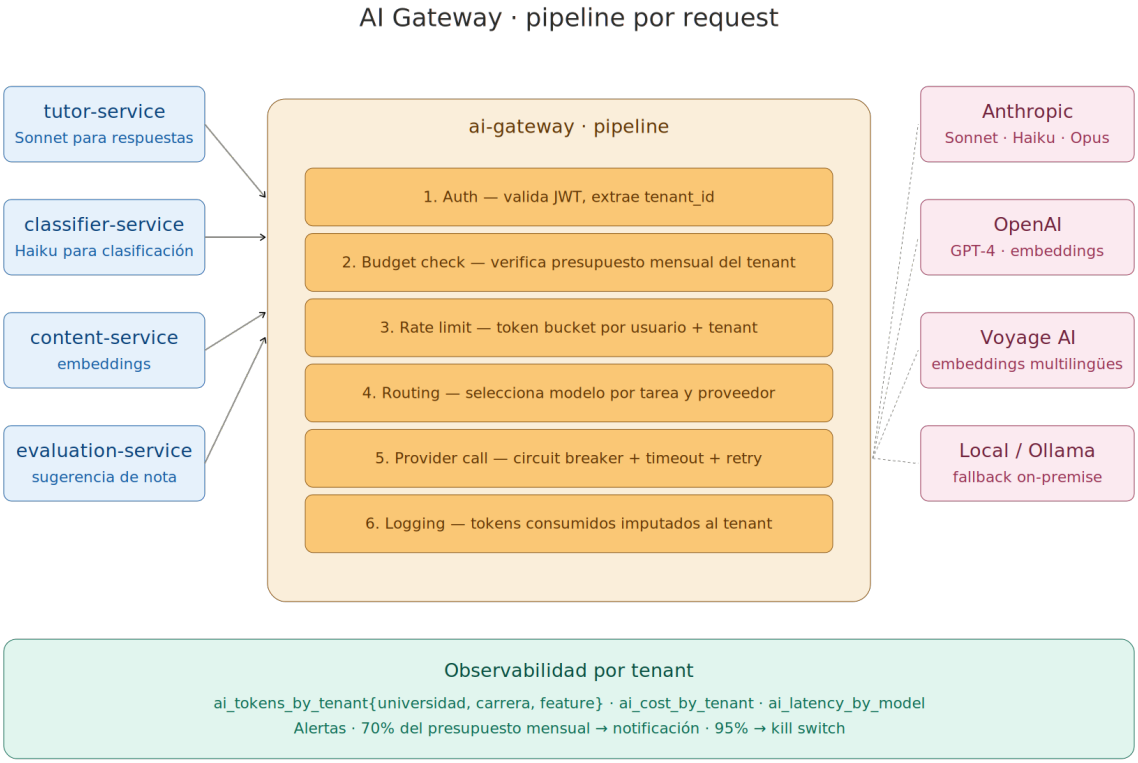

Figura 5: Pipeline del AI Gateway por cada request entrante: primero valida JWT y extrae tenant_id; luego verifica presupuesto mensual del tenant; aplica rate limit por usuario y tenant; rutea al modelo adecuado según el tipo de tarea (Sonnet para tutor, Haiku para clasificación, embeddings para RAG); invoca al proveedor con circuit breaker y retry; finalmente registra los tokens consumidos imputados al tenant. Observabilidad granular por universidad, carrera y feature.

`web-teacher` — Para `docente`. Dos secciones grandes balanceadas: *autoría* (crear contenido) y *seguimiento* (ver qué pasa con sus estudiantes). Editor de rúbricas con preview del cálculo en tiempo real; generador de ejercicios tipo formulario que produce JSON estructurado; gestor de base de conocimiento con visualización de qué hay indexado; dashboard con alertas priorizadas; vista de corrección asistida con la rúbrica, el código del estudiante y el CTR colaterales.

`web-student` — Para `estudiante`. Es la aplicación técnicamente más compleja porque aloja el IDE y el tutor con streaming. Tres vistas principales:

- *Home*: próximas actividades con su calendario, resultados recientes, mensajes del docente, progreso general.
- *Trabajo en problema*: layout de tres paneles redimensionables — enunciado y tests a la izquierda, editor Monaco en el centro, panel del tutor socrático a la derecha.
- *Mi proceso*: visualización pedagógica (no técnica) del propio CTR y del perfil de coherencia, con retroalimentación formativa automática.

1.10.2 Flujo central del estudiante

El flujo donde vive el valor diferencial de la plataforma es la resolución asistida de un problema. Cada evento del CTR se genera automáticamente sin que el estudiante lo perciba; el estudiante ve una experiencia fluida, el sistema registra una trayectoria auditable.

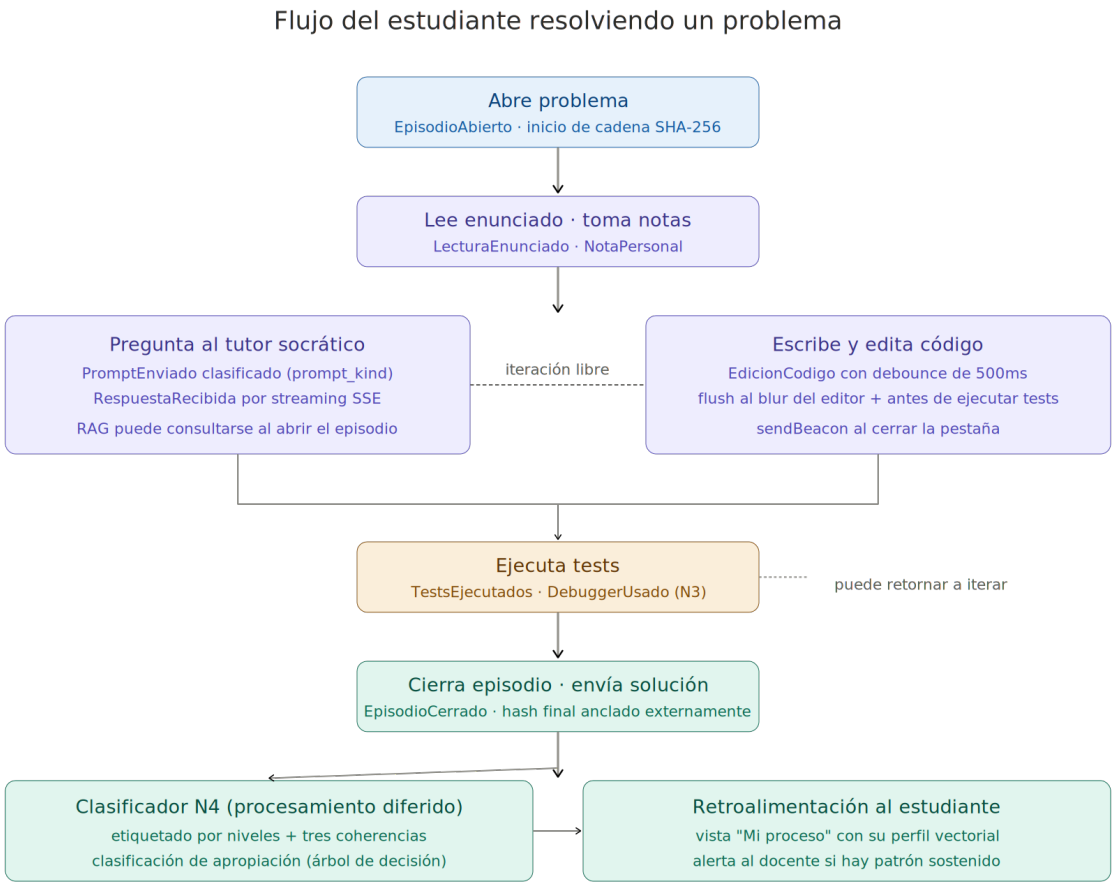

Figura 6: Flujo del estudiante resolviendo un problema: abre el problema y se emite EpisodioAbierto como inicio de cadena SHA-256; lee enunciado y toma notas; itera entre preguntar al tutor (con clasificación de prompt_kind y respuesta por streaming SSE) y escribir código (con debounce de 500 ms y flush al blur del editor); ejecuta tests (pudiendo volver a iterar); cierra el episodio emitiendo EpisodioCerrado con el hash final. El clasificador N4 procesa el episodio de manera diferida; la retroalimentación vuelve al estudiante en la vista “Mi proceso” y al docente como alerta si corresponde.

Detalles de implementación que importan:

- La emisión del evento `EdicionCodigo` tiene debounce de 500 milisegundos (Monaco dispara 4-10 eventos/segundo sin eso), más flush al blur del editor y antes de ejecutar tests. Esto registra "pausas significativas del estudiante" en vez de "cada tecla".
- La captura del evento `Abandono` usa `sendBeacon` en el `beforeunload` del navegador, porque es la única API que garantiza entrega cuando el usuario cierra la pestaña.
- Los eventos `PromptEnviado` y `RespuestaRecibida` no los emite el frontend — los emite el backend del tutor. El frontend solo renderiza la UX. Esto impide que un estudiante con DevTools manipule la cadena del CTR.
- El streaming del tutor se implementa sobre Server-Sent Events con parsing de eventos delimitados por doble salto de línea. La reconexión automática de SSE maneja caídas transitorias sin cortar la conversación.

1.10.3 Dashboard docente

El dashboard del docente es el punto de contacto con la analítica producida por el clasificador. Tiene **cuatro zonas funcionales** que responden a necesidades distintas:

Métricas del curso (fila superior, una línea). Operacional, cero cognición: estudiantes activos, episodios cerrados hoy, integridad del CTR, alertas pendientes. La métrica de integridad debe ser prominente — si cae, el dashboard mismo pierde confiabilidad evaluativa.

Distribución de apropiación (la señal pedagógica agregada). Barra apilada con porcentajes de estudiantes en cada categoría (delegación pasiva, apropiación superficial, apropiación reflexiva) para la comisión completa en la ventana seleccionable (semana, mes, período completo).

Feed de alertas (mecanismo de escalamiento). Tres tipos que se tratan distinto:

- *Patrón sostenido* (delegación persistente): diagnóstico formativo, requiere intervención docente tipo "charla guiada".
- *Intento repetido de jailbreak*: puede ser exploración legítima; el docente ve el contexto antes de intervenir.
- *Progresión positiva*: tan importante como las negativas. Amplifica el aprendizaje con retroalimentación docente confirmatoria.

Tabla individual con los tres componentes del vector de coherencia mostrados por separado. La tentación de colapsar las tres en un "score total" se resiste explícitamente: el perfil vectorial es lo pedagógicamente productivo, el número único oculta información.

1.10.4 Stack y decisiones de frontend

Todas las aplicaciones comparten el stack: React 19 con TypeScript, Vite para bundling, TanStack Query para gestión de estado remoto (caché, refetch, optimistic updates), TanStack Router para routing, Zustand para estado local cuando hace falta (casi nunca fuera del editor), Tailwind CSS v4 con shadcn/ui como biblioteca base, Recharts para gráficos, Monaco Editor para IDE.

La librería compartida `@platform/ui` contiene: componentes de diseño (botones, inputs, cards con el mismo DS), hooks para autenticación Keycloak unificada, cliente HTTP tipado con validación Zod, componentes de visualización del CTR, y el cliente del bus de eventos del frontend al CTR.

El deployment es estático: cada app se buildea a archivos estáticos servidos por CDN o por Nginx, con un único punto de configuración de runtime (`config.json` con la URL del API Gateway y los client IDs de Keycloak).

1.11 Integración de IA: tutor, RAG y generador

La plataforma integra inteligencia artificial en tres lugares con propósitos y restricciones distintos.

1.11.1 Tutor socrático

El tutor socrático es el componente más sensible de la plataforma desde el punto de vista pedagógico y el más diseñado desde el punto de vista técnico. Está completamente especificado en el documento *Sistema AI-Native N4 — Guía de implementación técnica*. Sus propiedades clave desde la perspectiva de la plataforma:

- **Prompt global por MAJOR, personalizable por MINOR.** El prompt del sistema es global para toda la plataforma en su versión MAJOR (1.x.x); las universidades pueden proponer ajustes MINOR (1.1.0 para UNSL, 1.2.0 para UNCuyo) que ajustan tono o ejemplos sin cambiar la postura socrática. Todos los MINOR se validan contra un schema de invariantes.
- **Versionado como primera clase.** Cada evento del CTR lleva el `prompt_system_hash` activo al momento del evento. Esto garantiza reproducibilidad aunque el prompt evolucione.
- **Modelo LLM comercial de alta capacidad.** Claude Sonnet como modelo primario por calidad socrática y soporte de SSE. Haiku para tareas auxiliares (clasificación de `prompt_kind`) por costo.

1.11.2 RAG por comisión

El RAG (Retrieval Augmented Generation) permite al tutor anclar sus respuestas al material del curso. No convierte al tutor en un chatbot de preguntas y respuestas — el tutor sigue respondiendo socráticamente, pero con material correcto.

Modo pasivo. Por defecto, antes de cada invocación al LLM, el sistema ejecuta un retrieval sobre el prompt del estudiante y las últimas respuestas del tutor, y el contexto relevante se inserta en el system message. El tutor puede entonces referenciar "lo que vimos en la unidad 3" con precisión.

Modo explícito. El docente puede marcar un problema como "requiere material X"; el retrieval se ejecuta al abrir el episodio y el contexto queda fijado para toda la sesión. Útil cuando hay documentación crítica que debe estar siempre presente.

Scoping estricto por comisión. El filtro `WHERE comision_id = :id` es parte de toda query al vector store. Un estudiante de Programación 2 nunca recibe material de Bases de Datos I, ni siquiera por un bug de aplicación, porque el filtro está en el índice y en la consulta.

Auditoría. Cada query al RAG genera un evento con qué chunks fueron recuperados y sus scores. Esto permite reconstruir qué información vio el tutor al momento de producir cada respuesta. El hash de los chunks recuperados forma parte del hash de configuración del evento CTR.

1.11.3 Generador de ejercicios

El generador permite al docente describir en lenguaje natural qué tipo de ejercicio quiere (*"tres ejercicios de recursión con memoización, dificultad CS2"*) y obtiene un JSON estructurado con enunciado, casos de prueba, rúbrica sugerida y tier.

No es un chat. Es un formulario que pide parámetros estructurados (tema, técnica, casos límite a cubrir, ejemplos análogos) y produce un draft. El docente revisa, edita cualquier parte, y recién cuando aprueba el ejercicio se materializa en la base.

Trazabilidad del ejercicio al prompt. Cada ejercicio generado queda linkeado al prompt y al contexto que lo produjeron, para auditoría e iteración del propio generador.

Refinamiento por comisión. Cada comisión puede tener un "preámbulo" del generador específico (*"siempre incluir al menos un caso con entrada vacía"*, *"usar identificadores en inglés"*), que se concatena al prompt global antes de la generación.

1.11.4 AI Gateway

Todas las invocaciones a LLMs y embeddings pasan por el `ai-gateway`. Su responsabilidad central es **convertir el costo de IA en una métrica observable y controlable por tenant**.

Sin el gateway, cada universidad es un riesgo financiero: un bug o un uso abusivo puede consumir miles de dólares de API en horas.

Propiedades del gateway:

- **Presupuesto por tenant y por carrera** con hard limits mensuales. Alerta al 70%, kill-switch al 95%, bloqueo total al 100%.
- **Rate limiting por usuario y por tenant** con token buckets para absorber ráfagas legítimas sin permitir abuso sostenido.
- **Routing por tarea** a modelos de costo/capacidad apropiados: Sonnet para el tutor (requiere calidad), Haiku para clasificación de `prompt_kind` (requiere volumen barato), embeddings para RAG.
- **Fallbacks entre proveedores** con circuit breakers: si Anthropic tiene degradación, el gateway puede rutear temporalmente a OpenAI (con marca explícita en el evento del CTR que el modelo usado fue distinto, para no contaminar análisis estadísticos).
- **Caché de respuestas idempotentes**: las clasificaciones con temperatura 0 se cachean por input+modelo+versión; ahorra ~40% del volumen de clasificación diferida.

Métricas granulares que expone: `ai_tokens_by_tenant{universidad, carrera, feature, modelo}`, `ai_cost_by_tenant_monthly`, `ai_latency_by_model_percentile`. Cada universidad tiene su propio dashboard de consumo.

1.12 Datos: persistencia y estrategia

1.12.1 Bases de datos

La plataforma opera con tres bases lógicas PostgreSQL 16, separadas pero en el mismo cluster:

`academic_main` — Dominio académico, evaluación, agregados analíticos, contenido. Tablas principales: `universidades`, `facultades`, `carreras`, `planes`, `materias`, `periodos`, `comisiones`, `inscripciones`, `usuarios_comision`, `problemas`, `actividades`, `rubricas`, `correcciones`, `calificaciones`, `materiales`, `chunks` (con embeddings pgvector), `chunks_metadata`. Cada tabla tiene columna `tenant_id` y política RLS activa.

`ctr_store` — Eventos del CTR, clasificaciones, anotaciones de validación Kappa. Tablas principales: `ctr_events` (particionada por mes, BRIN sobre timestamp), `ctr_anchors` (hash final externamente anclado), `classifications` (append-only con `is_current` booleano), `annotation_batches`, `annotation_items`, `annotations` (para el protocolo Kappa).

`identity_store` — Mapping pseudónimo-identidad, consentimientos, historial de acceso a datos personales, registro de derechos GDPR-style ejercidos. Acceso restringido al `identity-service`.

1.12.2 Schemas clave

Tabla `ctr_events` (simplificada):

```sql
CREATE TABLE ctr_events (
    id BIGSERIAL,
    tenant_id UUID NOT NULL,
    episode_id UUID NOT NULL,
    event_uuid UUID NOT NULL UNIQUE,
    seq INTEGER NOT NULL,
    event_type TEXT NOT NULL,
    ts TIMESTAMPTZ NOT NULL,
    payload JSONB NOT NULL,
    prompt_system_hash CHAR(64) NOT NULL,
    prompt_system_version TEXT NOT NULL,
    classifier_config_hash CHAR(64) NOT NULL,
    self_hash CHAR(64) NOT NULL,
    chain_hash CHAR(64) NOT NULL,
    PRIMARY KEY (id, ts)
) PARTITION BY RANGE (ts);
```

```sql
CREATE INDEX ON ctr_events (episode_id, seq);
CREATE INDEX ON ctr_events (tenant_id, ts DESC);
CREATE INDEX ON ctr_events (event_type) WHERE event_type IN ('EpisodioCerrado',
 ↪  'AlertaPedagogica');

-- Política RLS
CREATE POLICY tenant_isolation ON ctr_events
  USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

Tabla `comisiones`:

```sql
CREATE TABLE comisiones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    materia_id UUID NOT NULL REFERENCES materias(id),
    periodo_id UUID NOT NULL REFERENCES periodos(id),
    codigo TEXT NOT NULL,
    curso_config_hash CHAR(64),  -- hash del prompt + reference_profile
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (tenant_id, materia_id, periodo_id, codigo)
);

CREATE INDEX ON comisiones (tenant_id);
CREATE POLICY tenant_isolation ON comisiones
  USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

Tabla `classifications` (append-only):

```sql
CREATE TABLE classifications (
    id BIGSERIAL PRIMARY KEY,
    tenant_id UUID NOT NULL,
    episode_id UUID NOT NULL,
    classifier_config_hash CHAR(64) NOT NULL,
    appropriation TEXT NOT NULL,
    appropriation_reason TEXT NOT NULL,
    ct_summary FLOAT,
    ccd_mean FLOAT,
    ccd_orphan_ratio FLOAT,
    cii_stability FLOAT,
    cii_evolution FLOAT,
    classified_at TIMESTAMPTZ NOT NULL,
    is_current BOOLEAN NOT NULL DEFAULT true,
    UNIQUE (episode_id, classifier_config_hash)
);

CREATE INDEX ON classifications (episode_id, is_current) WHERE is_current;
```

1.12.3 Estrategia de particionado y archivado

La tabla `ctr_events` se particiona por mes (`ts`). Particiones antiguas (más de 18 meses) se archivan a almacenamiento frío S3-compatible con compresión `zstd`; el script de archivado verifica la integridad de la cadena antes de mover. La tabla materializada `episode_summary` mantiene un resumen por episodio que sigue disponible para dashboards cuando las particiones fuente están archivadas.

Las tablas del plano académico no requieren particionado para el volumen previsto del pilotaje (unas pocas universidades con miles de estudiantes). Si el volumen crece significativamente, `inscripciones` y `correcciones` son candidatas naturales a particionar por `periodo_id`.

1.12.4 Backups y recuperación

- Backups físicos con `pg_basebackup` diarios + archivo WAL continuo a S3-compatible.
- Retención de 30 días rolling + un backup mensual retenido 12 meses + un backup anual.

- Tests de restauración mensuales en entorno de staging con verificación automatizada (el backup restaurado pasa los tests de integridad del CTR).
- RPO objetivo: 15 minutos (pérdida máxima en caso de fallo). RTO objetivo: 4 horas (tiempo máximo de restauración).

1.13 Observabilidad y operación

1.13.1 Stack de observabilidad

Todos los servicios emiten telemetría a través de OpenTelemetry: traces distribuidos con correlation ID que atraviesa todos los saltos (frontend → API Gateway → servicios → bus → workers), métricas estructuradas con labels de tenant, logs estructurados en JSON con campo `trace_id`.

- **Traces**: OpenTelemetry Collector → Jaeger. Retención de 7 días para traces completos, 90 días para sampling reducido.
- **Métricas**: OpenTelemetry Collector → Prometheus → Grafana. Retención de 30 días para granularidad completa, 1 año para series agregadas.
- **Logs**: Structured logs → Loki → Grafana. Retención de 14 días para logs operacionales, indefinida para logs de auditoría.
- **Errores**: Sentry integrado en todos los servicios y frontends, con sourcemaps para el frontend.

1.13.2 SLO clave

Los SLO se miden por tenant y por feature, con presupuesto de error explícito:

SLO	Objetivo	Ventana	Penalización al breach
Disponibilidad API Gateway	99.5%	30 días rolling	pause de deploys
Latencia tutor (primer token)	p95 < 2s	7 días	review de perf
Latencia tutor (respuesta completa)	p95 < 5s	7 días	review de perf
Tasa de integridad CTR	100%	permanente	alerta P0 inmediata
Latencia dashboards	p95 < 3s	7 días	optimización de queries
Pérdida de eventos a DLQ	0%	7 días	investigación inmediata

1.13.3 Dashboards operativos

Grafana tiene cinco dashboards predefinidos para operación:

Dashboard "Plataforma" — Vista global: tenants activos, estudiantes concurrentes, latencia P95/P99 por servicio, tasa de errores 5xx, consumo de IA por tenant, lag de Redis Streams por partición.

Dashboard "CTR" — Núcleo auditable: integridad de cadenas (debe ser 100%), throughput de eventos/segundo, tamaño de DLQ (debe ser 0), lag del worker por partición, tiempo entre `EpisodioCerrado` y clasificación completa.

Dashboard "Tutor" — Calidad pedagógica: latencia primer token, latencia respuesta completa, `socratic_compliance` promedio, tasa de detección de jailbreak, distribución de `prompt_kind`.

Dashboard "IA por tenant" — Consumo y costo: tokens consumidos por universidad, carrera y feature; costo USD proyectado mensual; porcentaje de presupuesto consumido; distribución por modelo (Sonnet vs Haiku vs embeddings).

Dashboard "Infraestructura" — Recursos: CPU/memoria por pod, conexiones PostgreSQL, tamaño de las bases, cola de backups, certificados TLS próximos a expirar.

1.13.4 Runbook de incidentes

El repositorio mantiene un runbook operacional (`docs/operations/runbook.md`) con procedimientos paso a paso para incidentes típicos. Ejemplos:

- **Integridad de cadena rota** (P0): procedimiento para identificar el evento problemático, marcar el episodio como `integrity_compromised=true`, excluirlo de análisis estadísticos, abrir issue con timeline.
- **Evento a DLQ** (P1): análisis de causa, reintento manual si es transitoria, decisión de marcar episodio como comprometido si no es recuperable.
- **Presupuesto de IA al 95%**: procedimiento para notificar al `docente_admin`, evaluar si se habilita extensión temporal o se activa el kill-switch.
- **Degradación del tutor**: procedimiento de fallback entre proveedores, con marca explícita en los eventos del CTR.

1.13.5 Deployment y ambientes

Tres ambientes: **development** (local con Docker Compose), **staging** (VM institucional con infraestructura igual a producción pero datos de prueba), **production** (VM o cluster Kubernetes según volumen). El CI/CD despliega automáticamente a staging en cada merge a `main`, y a producción con aprobación manual después de al menos 24 horas en staging sin incidentes.

El patrón de deploy es **blue-green** para los servicios del plano académico (intercambio atómico entre versiones) y **rolling** para los workers del CTR (cada worker de partición se reinicia uno a la vez para no interrumpir el bus). Las migraciones de esquema se ejecutan antes del deploy con Alembic y son siempre backward-compatible con el código anterior (para poder hacer rollback sin migrar hacia atrás).

1.14 Plan de desarrollo por fases

El plan abarca siete fases a lo largo de dieciséis meses. El cronograma está dimensionado para ser ejecutable por un equipo reducido (doctorando como líder técnico más uno o dos colaboradores) sin requerir inversión inicial en infraestructura de escala empresarial. Cada fase cierra con un release en staging y un hito verificable.

1.14.1 F0 — Fundaciones (meses 1-2)

Objetivo: Infraestructura base sobre la que el resto de las fases se construye.

Entregables:

- Monorepo configurado con pnpm workspaces (apps: `api`, `web-admin`, `web-teacher`, `web-student`; libraries: `@platform/ui`, `@platform/contracts`).
- PostgreSQL 16 con las tres bases lógicas (`academic_main`, `ctr_store`, `identity_store`) creadas por migraciones Alembic.
- Row-Level Security habilitado por defecto en toda tabla creada. Helper SQL para crear tablas con RLS automático.
- Keycloak desplegado con dos realms de prueba y federación mock.
- Pipeline de CI con GitHub Actions: lint, type-check, tests unitarios, tests de integración, migraciones en dry-run, build de imágenes Docker.
- Pipeline de CD a staging automático en merge a `main`.
- OpenTelemetry Collector, Jaeger, Prometheus, Grafana, Loki y Sentry desplegados con configuración básica.
- `docker-compose.yml` operable para desarrollo local con una sola línea de comando.
- ADRs 1 a 8 escritos y reviewados en formato MADR.

Hito de cierre: un endpoint `/health` protegido por Keycloak que devuelve el tenant del JWT autenticado, con un test de integración que verifica que un usuario del tenant A no puede

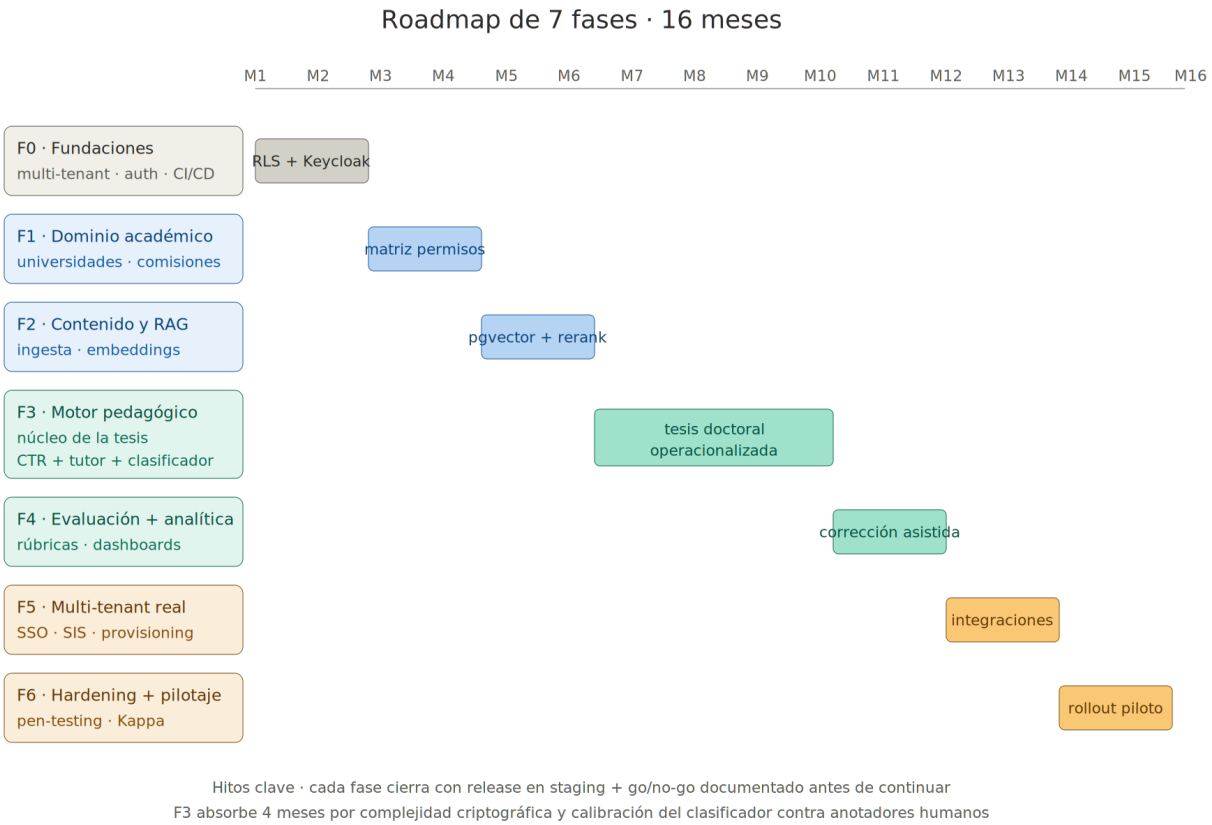

Figura 7: Roadmap de 7 fases en 16 meses: F0 Fundaciones (M1-M2) con multi-tenant, auth y CI/CD; F1 Dominio académico (M3-M4); F2 Contenido y RAG (M5-M6); F3 Motor pedagógico (M7-M10), la fase más larga y crítica que operacionaliza el núcleo de la tesis; F4 Evaluación y analítica (M11-M12); F5 Multi-tenant real con SSO y SIS (M13-M14); F6 Hardening y pilotaje institucional (M15-M16).

acceder a datos del tenant B ni siquiera inyectando el ID del tenant B en el request (la política RLS bloquea).

1.14.2 F1 — Dominio académico (meses 3-4)

Objetivo: gestión completa de la estructura académica con matriz de permisos implementada y validada.

Entregables:

- CRUD completo para universidades, facultades, carreras, planes de estudio, materias, comisiones, períodos lectivos e inscripciones en el `academic-service`.
- `enrollment-service` con importación masiva por CSV y validaciones de integridad.
- Frontend `web-admin` con vistas para `superadmin` y `docente_admin`: dashboard de gestión, formularios de creación/edición, tablas con filtros, acciones bulk.
- Matriz de permisos implementada con Casbin. Middleware de autorización en todos los endpoints.
- Suite de tests de integración que cubre el 100% de las celdas de la matriz de permisos (cada combinación rol × recurso × acción tiene un test).
- OpenAPI auto-generado desde Pydantic; documentación interactiva accesible.

Hito de cierre: un `docente_admin` de una universidad de prueba crea la estructura completa de una carrera ficticia (3 materias, 2 comisiones por materia, 30 estudiantes matriculados por CSV), un `docente` asignado solo ve sus comisiones, y los tests automatizados verifican que nadie fuera del rol asignado puede leer ni modificar.

1.14.3 F2 — Contenido y RAG (meses 5-6)

Objetivo: base de conocimiento por comisión con retrieval contextual de calidad.

Entregables:

- `content-service` con pipeline de ingesta multi-formato: PDF (via unstructured.io), Markdown, código (ZIP descomprimido y chunked con tree-sitter), video (transcripción Whisper).
- Chunking semántico con configuración por tipo de material (funciones Python completas, secciones markdown, páginas PDF con contexto).
- Embeddings con `multilingual-e5-large` local y fallback a Voyage AI.
- Persistencia en `pgvector` con índices IVFFlat.
- Retrieval con filtro por `comision_id` y re-ranking de los top-20 con `bge-reranker-base` para retornar top-5.
- Interfaz en `web-teacher` para cargar material, ver el índice por comisión, diagnosticar calidad del retrieval con queries de prueba.
- Auditoría de cada query al RAG: qué chunks fueron recuperados con qué scores, almacenado para trazabilidad.

Hito de cierre: un docente carga el programa de una materia y tres apuntes PDF (unas 50 páginas en total). Desde una batería de 20 queries curadas (preguntas típicas que un estudiante haría), el sistema recupera los chunks correctos con precisión subjetiva mayor al 80% según evaluación manual del propio docente.

1.14.4 F3 — Motor pedagógico (meses 7-10) — núcleo de la tesis

Objetivo: operacionalizar completamente el diseño descrito en la tesis y en el documento técnico complementario.

Entregables (todos específicos del núcleo de la tesis):

- `ctr-service` con cadena SHA-256 encadenada, outbox transaccional con Redis Streams particionado en 8 (workers single-writer por partición), advisory lock por episodio, idempotencia con `event_uuid` único, DLQ tras 3 reintentos, anclaje externo del hash final.
- `tutor-service` con prompt v1.0.0 versionado, hash SHA-256 computado al bootstrap y verificado contra el declarado (fail-loud), streaming SSE, guardarraíles pre-check (detección de jailbreak) y post-check (`socratic_compliance`), clasificación de `prompt_kind` con LLM auxiliar.

- Frontend `web-student` con layout de tres paneles, Monaco Editor integrado con debounce y beacon, panel de tutor con streaming SSE bien parseado.
- `classifier-service` con etiquetador N1-N4 (reglas heurísticas + LLM fallback), tres analizadores de coherencia (temporal con reference_profile, código-discurso con segment-then-align, inter-iteración), clasificador de apropiación con árbol de decisión explicable.
- Generador de ejercicios con schema JSON estructurado.
- `governance-service` con repo Git separado, commits firmados GPG, `/health` que expone los hashes activos.

Hito de cierre: dos docentes hacen pilotaje interno en paralelo (una carrera de UNSL y una segunda carrera). Los episodios cerrados pasan verificación de integridad del CTR al 100%. El clasificador produce clasificaciones que los docentes consideran coherentes con su juicio experto en una revisión manual de 30 episodios aleatorios.

1.14.5 F4 — Evaluación y analítica (meses 11-12)

Objetivo: rúbricas, corrección asistida y dashboards en pleno funcionamiento.

Entregables:

- Sistema de rúbricas con JSON Schema versionado, editor visual en `web-teacher`, preview del cálculo en tiempo real.
- `evaluation-service` con corrección asistida: sugerencia de puntaje por criterio a partir de código entregado, tests pasados, y evidencia del CTR.
- Frontend de corrección: rúbrica a un lado, código y CTR del otro, con controles para confirmar o ajustar cada puntaje.
- Dashboard docente con las cuatro zonas descritas (métricas del curso, distribución de apropiación, feed de alertas, tabla individual con las tres coherencias separadas).
- Dashboard del estudiante con vista “Mi proceso” pedagógica, no técnica.
- Exportación de reportes PDF para actas institucionales.
- Tres tipos de alertas activos (patrón sostenido, intento de jailbreak, progresión positiva) con propagación al plano académico vía eventos `AlertaPedagogica`.

Hito de cierre: una comisión piloto completa un cuatrimestre en el sistema. Las notas exportadas son consistentes con lo que el docente hubiera puesto en papel según su propia revisión, pero con trazabilidad completa de cada puntaje al CTR que lo sustenta.

1.14.6 F5 — Integración multi-tenant real (meses 13-14)

Objetivo: la plataforma deja de ser mono-institucional de prueba y pasa a operar genuinamente con múltiples universidades.

Entregables:

- Federación Keycloak con SAML u OIDC del IdP institucional, probada con al menos dos universidades reales o sus entornos de staging.
- Conector con SIS institucional para importación de inscripciones: modo batch diario o API en tiempo real según el sistema de cada universidad.
- Wizard de provisioning de nuevo tenant: configura realm Keycloak, políticas RLS iniciales, prompt semilla heredado del MAJOR global, reference_profile seed por tier, dashboards base, presupuesto de IA inicial.
- Presupuestos de IA por tenant en `ai-gateway` con hard limits y alertas.
- Dashboards agregados cross-tenant para `superadmin` con datos anonimizados.
- Derechos GDPR-style implementados en `identity-service`: export portable de datos del estudiante, delete con pseudonimización irreversible.
- Gestión de consentimientos versionada.

Hito de cierre: dos universidades distintas corriendo en la misma instancia, cada una con su login institucional funcional, cada una con su presupuesto de IA activo, sin ver datos de la otra en ningún endpoint, cada una con su dashboard separado, y un `superadmin` que ve agregados cross-tenant sin poder identificar individuos.

1.14.7 F6 — Hardening y pilotaje institucional (meses 15-16)

Objetivo: endurecimiento de seguridad, validación formal del clasificador y rollout controlado.

Entregables:

- Penetration testing externo: mínimo OWASP Top 10, escalamiento horizontal y vertical entre tenants, fuzzing de endpoints, revisión de RLS.
- Validación formal del clasificador N4: muestra estratificada de 200 eventos (50 por nivel), anotación por dos anotadores humanos en interfaz pseudonimizada, cómputo de Kappa de Cohen con bootstrap CI, análisis de discrepancias.
- Tests adversariales en CI: batería de personas adversarias (jailbreak por ficción, urgencia emocional, instrucciones conflictivas) corriendo contra el tutor en cada merge.
- Documentación completa de operación (runbook de incidentes, procedimientos de backup/restore, alertas configuradas, SLO).
- SLO/SLA escritos y comunicados institucionalmente.
- Onboarding guides para docentes y estudiantes.
- Rollout piloto en al menos dos universidades con cohortes reales.

Hito de cierre: rollout piloto con al menos 200 estudiantes activos, al menos 4 semanas sin incidentes P0, $\kappa \geq 0.70$ en validación del clasificador contra ambos anotadores humanos, pen-test passed sin hallazgos críticos.

1.14.8 Continuidad post-F6

Después del rollout piloto, el trabajo se organiza en ciclos de mejora continua de 2 meses con las siguientes líneas:

- **Refinamiento del prompt**: análisis de guardarraíles que fallan, iteración MINOR con re-validación, progresión a nueva MAJOR si hay hallazgos significativos.
- **Calibración del `reference_profile`**: re-calibración con datos empíricos del pilotaje, versionado apropiado, re-evaluación append-only de episodios afectados.
- **Nuevas carreras y universidades**: onboarding de instituciones adicionales con el wizard de provisioning.
- **Features postergadas**: mobile app nativa, integración con videoconferencia, certificados digitales, etc.

1.15 Riesgos y mitigaciones

Tres riesgos cuya mitigación hay que establecer desde el día uno porque sus consecuencias son acumulativas y difíciles de revertir.

1.15.1 Explosión de complejidad del clasificador

Descripción. La tentación de colapsar las tres coherencias en un score único, la tentación de sustituir el árbol de decisión explícito por un modelo entrenado para “mejorar precisión”, y la tentación de calibrar los umbrales tenant a tenant hasta que todos los dashboards “se vean bien” degradan de forma acumulativa las propiedades que la tesis defiende: explicabilidad, validez estadística, reproducibilidad.

Mitigación. El clasificador se declara explícitamente como árbol de decisión con regla y razón por rama, implementado en código Python versionado en `governance-service`. Cualquier cambio requiere PR con rationale escrito, aprobación de dirección de tesis para cambios MAJOR, y re-validación Kappa cuando el cambio afecta los umbrales centrales. Está explícitamente prohibido entrenar un modelo opaco sobre las clasificaciones producidas.

1.15.2 Drift del prompt del tutor entre universidades

Descripción. Si cada tenant personaliza libremente su prompt del sistema, en dos años habrá 15 prompts distintos con deriva semántica entre ellos, y los datos agregados perderán comparabilidad. Los dashboards cross-tenant se volverán engañosos.

Mitigación. El prompt del sistema es **global por MAJOR**, personalizable por MINOR. Las universidades pueden ajustar tono y ejemplos en su versión MINOR, no pueden cambiar la postura socrática. El `governance-service` valida el prompt de cada tenant contra un schema

de invariantes (reglas normativas que no se pueden remover, como *"nunca entregar código completo sin intento previo del estudiante"*). Los análisis estadísticos cross-tenant solo incluyen episodios clasificados con el mismo MAJOR activo.

1.15.3 Costo descontrolado de IA

Descripción. Sin controles, una universidad puede quemar miles de dólares de API en un mes si un docente mal intencionado, un script malicioso, o simplemente un bug de aplicación dispara millones de queries.

Mitigación. Hard limits por tenant en el AI Gateway, con presupuesto mensual configurable por `docente_admin` con valor por defecto conservador. Alertas tempranas al 70% del presupuesto. Kill-switch al 95% que bloquea nuevas invocaciones preservando las en curso. Budget por carrera, no solo por universidad (una carrera populosa no puede consumir el presupuesto de una pequeña). Dashboards de consumo accesibles al `docente_admin` en todo momento para autoregulación.

1.15.4 Otros riesgos a monitorear

Desgaste del equipo. Dieciséis meses es un cronograma exigente para un equipo reducido. Mitigación: hitos verificables por fase, capacidad de "graduar" el alcance (entregar F5 con SIS solo para una universidad si el tiempo apremia), comunicación explícita con dirección de tesis sobre expectativas.

Dependencia de proveedores de IA. Un cambio de precios o una degradación prolongada de Anthropic puede afectar la viabilidad. Mitigación: AI Gateway con fallback a otros proveedores, abstracción de modelo en los prompts para permitir swap, snapshot del prompt que funciona con el modelo declarado en el hash.

Cambio de políticas institucionales. Una universidad puede cambiar su política de datos o de IA en educación y forzar un cambio operacional. Mitigación: contratos claros desde el onboarding, capacidad de exportación completa de datos de un tenant, capacidad de eliminación total con auditoría.

1.16 Glosario

AI Gateway. Servicio centralizado por el que pasan todas las invocaciones a LLMs y embeddings. Centraliza autenticación, presupuesto, rate limiting, routing, observabilidad.

Apropiación (reflexiva / superficial / delegación). Tres categorías de uso del tutor por parte del estudiante, definidas en la tesis. Salida del clasificador de apropiación.

Cadena SHA-256 del CTR. Secuencia de eventos encadenados donde cada `chain_hash` depende del evento actual y del `chain_hash` anterior. Garantiza detección de manipulación.

CTR (Cognitive Trace Record). Registro completo e inmutable de la actividad de un estudiante en un episodio. Elemento central de la tesis.

Classifier_config_hash. Hash SHA-256 de la configuración completa del clasificador (reglas del etiquetador, reference_profiles por tier, umbrales). Forma parte del hash de cada clasificación para reproducibilidad.

Coherencia (temporal / código-discurso / inter-iteración). Las tres dimensiones de análisis propuestas en la tesis. Nunca se colapsan en un score único.

Comisión. Instancia concreta de una materia en un período lectivo, con sus docentes, estudiantes y configuración propia. Unidad operativa principal.

Docente administrador. Rol con permisos de gestión institucional dentro de una universidad. Configura la estructura académica y supervisa auditoría.

Episodio. Una sesión de un estudiante resolviendo un problema, con inicio (`EpisodioAbierto`) y fin (`EpisodioCerrado`) explícitos.

Kappa de Cohen. Coeficiente estadístico para medir acuerdo entre clasificaciones. Umbral $\geq$ 0.70 define validación aceptable del clasificador.

MAJOR / MINOR / PATCH. Versionado semántico del prompt del sistema. MAJOR implica cambios sustanciales que requieren nueva validación; MINOR permite personalización por tenant dentro de invariantes; PATCH son correcciones menores.

Multi-tenant. Arquitectura que aloja múltiples universidades aisladas en una misma instancia de la plataforma.

N1 a N4. Cuatro niveles cognitivos de la tesis: N1 comprensión del problema, N2 diseño de estrategia, N3 validación ejecutiva, N4 interacción con IA.

Prompt_kind. Clasificación epistemológica del prompt del estudiante (solicitud directa, comparativa, epistemológica, validación, aclaración de enunciado). Se asigna al momento de emisión.

RAG (Retrieval Augmented Generation). Técnica que enriquece la respuesta del LLM con fragmentos relevantes recuperados de una base documental.

Reference_profile. Perfil empírico de un episodio con "apropiación reflexiva" para un tier de problema. Define el patrón contra el que se mide coherencia temporal.

RLS (Row-Level Security). Mecanismo de PostgreSQL que filtra filas automáticamente según políticas declarativas. Base del aislamiento multi-tenant.

SSE (Server-Sent Events). Protocolo HTTP para streaming unidireccional del servidor al cliente. Usado por el tutor para respuestas en streaming.

Tenant. En la plataforma, una Universidad. La unidad de aislamiento multi-tenant.

Tier. Categorización de dificultad de un problema (cs1_easy, cs1_medium, cs2). Cada tier tiene su propio reference_profile.

1.17 Apéndice: ADRs completos

Los ADRs detallados en formato MADR (Markdown Any Decision Record) viven en el directorio `docs/adr/` del repositorio. La lista completa de ADRs iniciales:

ID	Título	Estado
ADR-001	Multi-tenancy por Row-Level Security	Aceptado
ADR-002	Keycloak como IAM central con federación	Aceptado
ADR-003	Separación de bases lógicas	Aceptado
ADR-004	AI Gateway propio	Aceptado
ADR-005	Redis Streams como bus de eventos	Aceptado
ADR-006	FastAPI + SQLAlchemy 2.0 en backend	Aceptado
ADR-007	React 19 + TanStack en frontend	Aceptado
ADR-008	Casbin para autorización fine-grained	Aceptado
ADR-009	Git como fuente de verdad del prompt	Aceptado
ADR-010	Append-only para clasificaciones	Aceptado
ADR-011	pgvector para RAG en MVP	Aceptado
ADR-012	Monorepo con pnpm workspaces	Aceptado
ADR-013	OpenTelemetry como estándar de observabilidad	Aceptado
ADR-014	Docker Compose para dev, Kubernetes para prod	Aceptado
ADR-015	Blue-green deployment para servicios académicos	Aceptado

Cada ADR sigue el template: contexto, decisión, consecuencias (positivas y negativas), alternativas consideradas, referencias.

1.18 Documentos relacionados

- **Sistema AI-Native N4 — Guía de implementación técnica.** Documento técnico complementario que describe en detalle el diseño del plano pedagógico-evaluativo (CTR, tutor socrático, clasificador N4, validación Kappa). Este documento de arquitectura referencia ese documento como unidad y se concentra en la plataforma institucional.

- **Tesis doctoral "Modelo AI-Native con Trazabilidad Cognitiva N4 para la Formación en Programación Universitaria".** Marco teórico, metodología, evidencia empírica y agenda confirmatoria que fundamentan todo el diseño.
- **Repositorio `ai-native-prompts`.** Prompts versionados con commits firmados GPG, custodiados por el `governance-service`.
- **Documentación operacional** (`docs/operations/`). Runbooks, procedimientos de backup, configuración de alertas.
- **ADRs** (`docs/adr/`). Decisiones arquitectónicas individuales en formato MADR.

Documento generado como formalización del plan de arquitectura de la plataforma académica multi-universidad, operacionalización institucional del sistema AI-Native N4 propuesto en la tesis doctoral de Alberto Alejandro Cortez.